

赵强

求职意向：前端开发 | 北京

年龄 27 岁

性别 男

工作年限 5 年经验

电话 15614372721

邮箱 q1252595984@163.com



教育背景

2018-06 ~ 2022-09

河北科技大学

电子科学与技术 (本科)

技能特长

- AI 赋能与自动化工程：具备将 AI 工具链深度融入前端开发流的实战经验；熟练使用 Node.js / NestJS 搭建 MCP (Model Context Protocol) 桥接服务，解决内网权限屏障；
- 擅长借助 Claude Code、AST 解析与 create-skills 工具，实现组件库 API 的自动化提取与 Cursor Skills 的全网下发，大幅提升团队研发效能与业务交付速度。
- 框架技术能力：精通 Vue 2/3 及其技术生态，深入理解其响应式原理、Diff 算法及模板编译等核心机制；具备基于 Vue 3 Composition API 开发复杂应用的能力。
- 工程化能力：熟悉 Webpack、Vite 等构建工具，熟悉其配置优化与构建性能差异；具备自定义 Loader/Plugin 的能力，擅长通过工程化手段提升开发效率与项目质量。
- 数据可视化：拥有丰富的企业级数据可视化开发经验，精通 ECharts、AntV 等库；具备二次封装复杂图表组件及定制化开发的能力，满足多样化的数据展示需求。
- SDK 应用与即时通讯能力：具备丰富的第三方 SDK 集成与二次封装经验；具备将底层 SDK 封装为可复用组件库的能力，能解决跨平台 SDK 集成、授权流程、消息可靠性等复杂技术难题。

工作经验

2024-09 ~ 至今

百融云创科技有限公司

前端开发工程师

- 负责团队 AI 智能化工程基建，独立开发基于 NestJS 的 MCP 桥接服务解决内网鉴权痛点，并自研自动化流水线，实现组件库 API 提取与 Cursor Skills 的全网自动下发。
- 负责私域运营系统开发，从 0-1 搭建构建项目，负责企微 Copilot 开发，企微聚合联调功能开发
- 负责聚合聊天即时通讯 OpenIM SDK 架构改造
- 参与公司 uxd-next 项目脚手架开发，实现 vix-components 业务组件库本地化

2021-12 ~ 2024-09

用友网络股份有限公司

前端开发工程师

- 负责 NCC 低代码开发，包括设计基本数据结构和部分功能优化
- 使用 qiankun 微前端开发用友金融人力资源管理系统，提高系统集成的灵活度
- 开发人力资源系统业务组建，制定统一规范，并针对常用业务场景提出解决方案

项目经历

AI 智能化前端工程底座 (AI Workflow & MCP)

核心开发者

项目描述：

为解决公司内部由于安全隔离（内网鉴权、Cookie 限制）导致的 AI 工具链（如 Cursor）无法读取内网文档，以及内部组件库 API 频繁更新导致 AI 生成代码容易报错的问题。自研基于 MCP 协议的桥接服务，并串联 Claude Code 与内部 CI/CD 平台，打造了从“需求解析 -> 源码提炼 -> Skills 生成 -> 全网下发”的完整 AI 自动化工作流，显著提升前端团队的 AI 编码准确率与研发效能。

技术栈：Node.js、NestJS、Puppeteer、MCP 协议、TypeScript

核心职责：

- 突破内网隔离的 MCP 服务：基于 NestJS 从 0 到 1 搭建 MCP 服务层。通过单例模式与 Promise 队列解决高并发请求下的鉴权问题，自动模拟登录并维系动态 Cookie，使外部 AI 模型能合规且稳定地读取、解析内网富文本及图文需求文档
- 源码级 AI 分析与组件 API 提取：构建基于 AST 与大模型结合的代码扫描引擎。在组件库 (uxd-ui) 发版时，自动触发 Claude Code 遍历分析组件的 TypeScript 类型定义、Props、Events 及 Slots，确保复杂泛型与接口定义的 100% 精准提取。
- Cursor Skills 自动化生成流：深度应用 Claude Code 内置的 skill-creator 工具，将提取出的代码结构化数据，自动化转化为标准化的 Cursor Skill 描述文件，包含组件使用场景、最佳实践及边界处理，大幅降低开发者查阅文档与手写 Prompt 的心智负担。
- 全网下发与灰度管理机制：打通内部工程化发布平台，构建“项目-组件库”的依赖树映射。将生成的 Skills 版本化，通过灰度发布与秒级回滚策略，精准下发至对应业务团队的 .cursor/rules 目录，实现业务侧开发者环境的无感同步更新。

项目描述：
基于开源 OpenIM 进行深度定制化改造，打造企业微信私域聚合聊天平台，核心实现主账号托管多企微账号能力，支持统一界面完成多账号好友管理、消息收发、会话隔离，解决原生 IM 单一对一场景无法适配多主体托管的业务痛点，支撑企业私域运营高效管理多账号沟通体系。

技术栈：Vue3、TS、UXD-UI、VIX-Components

核心职责：

- 虚拟账号体系：打破原生 IM 1:1 绑定限制，设计了基于 虚拟ID = 托管账号ID + 好友ID 的映射映射规则。在不大规模改动数据库结构的前提下，通过 ID 空间置换技术最大化复用原生单聊逻辑，规避频繁重连、消息丢失、多账号无法同时在线等等问题，并且重构成本降低 60%
- WASM 内核无侵入集成方案：针对后端改造的 OpenIM WASM 内核，设计了一套基于全局上下文的调用层。通过封装异步调度、异常捕获与 TypeScript 类型约束，实现了对 switchAccount 等核心接口的无感调用，避免了对原生 SDK 源码的 Fork 维护，确保了基座升级的可持续性
- 多维度会话隔离与状态管理：
 - 数据隔离：在会话扩展字段 ex 中植入托管身份标识，重写会话 ID 生成算法与路由分发逻辑
 - 状态同步：实现了消息流、未读计数、草稿箱在「主账号 + 托管身份」双维度的状态隔离，彻底规避了身份切换时的数据串号与缓存污染问题
- 消息链路优化：
 - 构建“激活身份 -> 虚拟切换 -> 消息透传”的标准化发送流水线，在发送/回执/撤回全链路中增加身份标识
 - 稳定性治理：放弃传统的“登入登出”切换方案，改为单 WebSocket 多路复用模式，解决了频繁重连导致的初始化失败、单账号独占等性能瓶颈，支持多托管账号同时在线实时收发

企业微信AI Copilot侧边栏智能助手

前端核心开发

项目描述：
该项目是嵌入企业微信会话场景的AI智能助手类H5应用，核心定位为员工提供会话全链路AI辅助，依托企微SDK与内部自研大模型，实现对话理解、智能响应、业务联动的Copilot核心能力，大幅提升一线员工客户服务效率与专业度，落地AI在企业级会话场景的实际应用。

技术栈：Vue3、TS、UXD-UI、ShadCN UI、企微SDK

核心职责：

- 企微生态深度集成与安全基建：从 0 到 1 搭建企微 JS-SDK 集成体系，设计了全流程权限认证方案（Config/AgentConfig），通过服务端签名预校验与环境自适应逻辑，保障了应用在企微/微信/外部联系人等多端环境下的合法运行与核心 API 调用稳定性
- AI 智能会话交互：
 - 流式交互设计：实现基于 SSE/WebSocket 的流式输出（打字机效果），通过前端缓冲区算法优化，解决了长文本截断与渲染闪烁问题，提升了阅读体验
 - 上下文感知与清洗：设计会话内容提取机制，利用正则与 NLP 预处理进行敏感信息脱敏与关键词处理，为大模型提供精准的 Prompt 上下文
- 数据驱动的动态业务看板：
 - 构建“意图触发”式渲染引擎：基于 AI 识别的客户意图，实时触发 Echarts 资产配置走势与理财产品匹配图表
 - 基于企微 SDK 实现了“图表转图片”并一键分发至当前会话的闭环功能，显著提升了投顾服务的专业度
- 自动化流程（Workflow）可控管理：将 AI 建议与内部工作流决策链联动。开发了包含“AI 自动派单、节点状态实时同步、人工介入干预”的复合型流程组件，确保了 AI 辅助场景下的业务合规性与流程闭环
- 工程化提效与调试方案：针对企微 H5 难以本地调试的痛点，搭建了基于 Whistle + HTTPS 代理的真机调试环境。通过 Mock 企微身份参数与 SDK 接口注入，提升联调效率

uxd-next

核心开发者角色

项目描述：
参考 shadcn/ui 「源码级本地安装、按需拉取」核心设计思想，自研企业级前端业务组件脚手架，解决内部多业务线组件复用混乱、定制化成本高、npm 包版本锁死等问题。脚手架采用组件与 CLI 解耦架构，支持业务组件独立打包、远程源码托管、TS/JS 自动兼容，通过命令行实现组件本地一键安装，组件更新无需迭代脚手架，支撑公司内部中后台、私域运营等多场景业务开发，提升组件复用率与开发效率超 60%

技术栈：Vue3 + TypeScript + Vite + Monorepo + AST + 命令行开发 + 静态资源托管

核心职责：

- 架构设计与解耦：采用 Monorepo 架构将组件库工程与 CLI 工具彻底分离。CLI 仅作为资源调度器，组件源码独立托管。实现组件更新与脚手架逻辑零耦合，支撑了新组件的“即发即用”
- 源码级构建体系：基于 Vite/Rollup 搭建单组件独立打包流水线。定制构建输出规则，保留 TSX 源码可读性，实现样式内聚化与路径别名（Alias）自动转换，确保拉取后的代码可直接在业务代码中修改定制
- 按需调度引擎：构建“远程源码托管 + Key-Map 映射”机制。通过维护结构化的资源索引表，支持脚手架精准拉取目标组件及其依赖，解决了传统包管理器无法按需获取源码的局限
- 核心CLI开发（add/build）：
 - add 命令：实现远程组件拉取、本地依赖检测及 TS 到 JS 的动态降级编译，确保组件无缝适配不同技术背景的项目环境
 - build 命令：自动化执行组件打包、哈希生成及远程映射表同步，实现组件库的持续集成与热更新
- 标准化规范：基于 Vue3 Composition API 开发首批高频业务组件。利用 CSS 变量（Variables）设计样式系统，兼顾了脚手架的内聚设计与业务端的定制化主题需求。